Intern 3-Month Training Program

Full Stack Engineer Internship

Program Duration	12 Weeks (3 Months)
Technology Stack	JavaScript · TypeScript · React.js · Node.js · Express.js · NestJS · SQL (PostgreSQL) · NoSQL (MongoDB) · Git & GitHub
Styling & UI	Tailwind CSS, shadcn/ui

Program Overview

This 3-month Full Stack Engineer Internship Program is designed to transform beginners into job-ready full-stack developers capable of building modern, scalable, and maintainable web applications.

The program follows a structured, phase-wise learning approach combining frontend UI development, backend API engineering, database design, and real-world best practices. Each phase focuses on hands-on learning through Proof of Concepts (POCs) and guided projects.

Technology Stack Covered

- **Frontend:** JavaScript, React.js, TypeScript
- **Backend:** Node.js, Express.js, NestJS, OOP concepts
- **Databases:** SQL (PostgreSQL) and NoSQL (MongoDB)
- **Styling & UI:** Tailwind CSS, shadcn/ui
- **Version Control:** Git and GitHub
- **APIs:** RESTful APIs, authentication and authorization

What Interns Will Learn

- Version control with Git and GitHub using team workflows
- Object-oriented programming principles with hands-on practice for every core concept
- JavaScript fundamentals applied through real POCs
- TypeScript for type-safe application logic
- React component architecture, hooks, and state management
- SQL querying, schema design, transactions, triggers, functions, and stored procedures
- Node.js backend development: user management, authentication, and product APIs as one combined project
- NestJS modular APIs, validation, and persistence
- End-to-end full-stack integration connecting React frontend with Node.js backend

---

Program Structure (Phases)

#	Phase	Timeline
1	Git, GitHub & Object-Oriented Programming	Days 1–2
2	JavaScript Fundamentals & POCs	Days 3–8
3	TypeScript Fundamentals and Practice	Days 9–10
4	React.js — Frontend Framework Training	Days 11–17
5	Database Learning — SQL & PostgreSQL	Days 18–19
6	Node.js Backend Development	Days 20–29
7	NestJS Backend Development	Days 30–39
—	Final POC — Full-Stack Integration	Days 40–41

Phase 1: Git, GitHub & Object-Oriented Programming

Timeline: Days 1–2

Day 1 — Git and GitHub

Objective: Learn version control fundamentals to manage code effectively and collaborate with teams.

🎯 Objective: Help QA team understand and use Git & GitHub confidently for collaboration and version control.

🧩 1. Purpose of Git & GitHub
💡 Why do we use version control?

✅ Keep track of code history
✅ Work together without overwriting each other's changes
✅ Roll back when things break
💬 Discussion Prompt:

Have you ever lost work because of overwriting a file or miscommunication in the team?

📌 Key Concepts:

Git = version control tool
GitHub = platform for hosting Git repositories (online)
⚙️ 2. Git Basics: Setup & Cloning
Download git https://git-scm.com/downloads

🔧 Setup Git:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

🔐 Using SSH for Authentication:

Generate SSH Key:
It will ask for location to save
Also ask for pasphrase - skip this step
ssh-keygen -t ed25519 -C "your_email@example.com"
- Add the key to GitHub: [GitHub SSH Settings]
📂 Clone a Repo:

```
git clone git@github.com:user/repo.git
```

🧪 Try This Live:

Clone a sample repo and navigate into it.

🌿 3. Branching & Merging
🌱 Create a Branch:

```
git checkout -b feature/test-automation
```

🔀 Merge Back into Main:

```
git checkout main
git merge feature/test-automation
```

💬 Team Activity:

Create your own branch and add a comment to a file. Merge it back and push.

🛠️ 4. Daily Commands for QA Work
🧩 Task 💻 Command
Check status git status
Stage a change git add filename
Commit a change git commit -m "Your message"
View log git log --oneline --graph
Save unfinished work git stash
Get back stashed work git stash pop
🤔 Tip: Commit often with clear messages.

🌍 5. Remote Operations
🔗 View remotes:

```
git remote -v
```

📥 Pull latest changes:

```
git pull origin main
```

📤 Push your changes:

```
git push origin feature/test-automation
```

🔄 Upstream vs Origin:

origin = your fork
upstream = original repo (for open-source or team forks)
🔥 6. Conflict Resolution
🚨 Scenario: Two people edited the same line.

Git shows a conflict during merge
Edit file to fix manually:
<<<<<<< HEAD

Your changes
=======
Incoming changes

> branch-name
> Save and commit resolved file:
> git add conflicted-file
> git commit -m "Resolved conflict"
> 🧠 Quick Exercise:

Simulate a conflict in pairs and resolve it.

🚀 7. Advanced Remote Commands: fetch, pull, push, and More
🔄 git fetch
Fetches the latest branches and commits from the remote, but does not merge them into your local branch.

git fetch origin
Useful When: - You want to inspect new commits before merging - You want to compare remote changes without affecting your working directory

📝 Example:

git fetch origin
git log HEAD..origin/main --oneline
📥 git pull
Combines git fetch and git merge. It pulls changes and merges them automatically.

git pull origin main
⚠️ Be Careful: If your local branch has diverged, this may create a merge commit or conflict.

📤 git push
Pushes your local commits to the remote repository.

git push origin feature/bug-fix-qa
Useful When: - You've committed locally and want others to see your changes - You want to create a Pull Request on GitHub

🔁 git pull --rebase
Pulls changes and applies your commits on top of what's new from remote.

git pull --rebase origin main
Pro: Creates a cleaner, linear history.
Con: Can be complex if conflicts occur.

🔧 9. Useful Git Commands for QA Tasks
Command Description
git diff Show changes not yet staged or committed
git diff --staged Show changes that are staged for the next commit
git log Show commit history
git log -p Show history with diffs
git show Show details of a specific commit
git reset --hard HEAD Reset current branch to last commit (careful: discards changes)
git clean -fd Remove untracked files and directories

git reflog See all recent HEAD movements (useful for recovery)
🧭 10. Final Pro Tips for QA
Use branches for every feature, test, or bugfix.
Pull often to avoid conflicts.
Use git stash before switching branches with uncommitted changes.
Always read commit messages and use git log before merging.
Prefer pull --rebase if your team uses a clean history policy.

Hands-On Tasks

1. Create a GitHub account with a professional username
2. Install Git on your computer
3. Configure Git with your name and email
4. Create the training repository (e.g. `fullStackInternTraining`)
5. Create folder structure for all upcoming projects
6. Make your first commit and push to GitHub
7. Practice making multiple commits with meaningful messages
8. Create a branch, make changes, merge it back, and push
9. Simulate a conflict with a partner and resolve it

Playground: [Learn Git Branching]

Repository Folder Structure

```
fullStackInternTraining/
├── README.md
├── Phase01_Git_OOP/
├── Phase02_JavaScript_POCs/
├── Phase03_TypeScript/
├── Phase04_React/
├── Phase05_SQL/
├── Phase06_Node/
├── Phase07_NestJS/
└── Final_Integration/
```

Day 2 — Object-Oriented Programming (OOP)

Objective: Develop a strong understanding of Object-Oriented Programming fundamentals and apply these principles in JavaScript/TypeScript applications.

Learning Resources

- Video (YouTube): [OOP Concepts]
- OOP Reference: [OOP Pillars — Scaler Topics]

Core OOP Concepts to Learn

- Class and Object
- Encapsulation

- Abstraction
- Inheritance
- Polymorphism
- Constructor and Destructor
- Access Modifiers (public, private, protected)
- Method Overloading and Overriding
- Interfaces (in TypeScript)

Practical Tasks (Mandatory — One Per Concept)

Concept	Practical Task
Class and Object	Create a `Product` class with properties (name, price, category) and a `User` class with properties (name, email, role). Create multiple objects from each class and call their methods to print details.
Encapsulation	Add a private `_password` field to the `User` class. Create getter and setter methods so the password can only be changed if the new value is at least 8 characters long. Try setting an invalid password and confirm it gets rejected.
Abstraction	Create an abstract class called `PaymentProcessor` with an abstract method `processPayment(amount)`. Then create two child classes — `CreditCardProcessor` and `UPIProcessor` — each implementing `processPayment()` differently. Call each from outside without knowing the internal logic.
Inheritance	Create a base class `Vehicle` with properties (make, model, year). Create two child classes — `Car` (adds numberOfDoors) and `Bike` (adds hasSidecar). Override a `describe()` method in each child class to print vehicle-specific details.
Polymorphism	Create a base class `NotificationChannel` with a `send(message)` method. Create three child classes — `ConsoleChannel`, `EmailChannel`, and `SMSChannel` — each with its own version of `send()`. Write a function that takes any `NotificationChannel` and calls `send()` — it should work correctly regardless of which channel type is passed.

Deliverable: Submit a PDF or Markdown file with notes from the video and article, concept summaries, and working code snippets for **each** practical task listed above. Code files should be committed under `Phase01_Git_OOP/`.

Note: OOP concepts are revisited in later phases when writing TypeScript types/classes, React component patterns, and Node/NestJS services.

Phase 2: JavaScript Fundamentals & POCs

Timeline: Days 3–8

Objective: Build a strong foundation in JavaScript by learning core language concepts and applying them through hands-on POC projects.

Learning Resource (Recommended)

- JavaScript sections from: [Introduction to HTML, CSS & JavaScript — IBM via Coursera]

Core JavaScript Topics

- Variables, data types, and operators
- Conditionals and loops
- Functions and scope
- Arrays and objects
- DOM manipulation (selecting elements, modifying content and styles)
- Event handling (click, submit, input)
- Form validation using JavaScript
- Debugging with browser developer tools

POC 1: Student Result Processor

What you will build: A system that takes student names and their subject marks, then automatically calculates each student's percentage and grade, and shows a summary report for the whole class.

What the system should do:

- Let the user enter student information: name and marks for each subject
- Check that the name is not empty and marks are numbers between 0 and 100
- Calculate the percentage for each student
- Assign a grade based on the percentage (e.g. A for 90+, B for 75–89, etc.)
- Show class-level statistics: average percentage, who scored highest, who scored lowest
- Display all results in a clean, readable format on the page
- Handle bad input gracefully — show helpful error messages instead of crashing

What you will learn: Working with arrays and objects, writing functions, using conditional logic, doing math calculations, and validating user input.

Deliverable: A working result processor that runs in the browser; code committed to your repo; demo to mentor.

POC 2: User Feedback & Contact Management System

What you will build: A web form where users can submit feedback or contact requests. The form checks all inputs before allowing submission and shows clear success or error messages.

What the system should do:

- Create a form with these fields: Full Name, Email, Phone Number, Feedback Type (dropdown: Inquiry / Support / Complaint / General), Message (text area), and a "I agree to terms" checkbox
- Validate every field before submission:
- Full Name must not be empty
- Email must be a valid format (use regex)
- Phone number must be a valid format (use regex)
- Message must have at least 20 characters — show a live character count
- The Submit button should stay disabled until the "I agree to terms" checkbox is checked
- On successful submission: show a green success message and reset the form
- On validation failure: show specific red error messages next to each invalid field

What you will learn: Selecting and modifying DOM elements, listening for user events (click, input, submit), writing validation logic with regex, and managing form state.

Deliverable: A fully working feedback form with all validations; code in your repo; demo to mentor.

POC 3: Product Listing & Cart System

What you will build: A mini online shop where products are displayed in a list, and users can add items to a shopping cart, change quantities, remove items, and see the total price update in real time.

What the system should do:

- Display a list of products loaded from a JavaScript array (each product has: name, price, description, and a placeholder image)
- Each product has an "Add to Cart" button
- When a product is added to the cart:
- It appears in a cart section on the page
- If the same product is added again, increase its quantity instead of adding a duplicate
- Inside the cart, users can:
- Increase or decrease the quantity of any item
- Remove an item entirely
- The cart shows: each item's name, quantity, line total (price × quantity), and the overall cart total
- **Optional:** Save the cart to `localStorage` so it survives page refreshes

What you will learn: Manipulating the DOM dynamically, managing application state in memory, using array methods (filter, map, reduce), handling events on dynamically created elements, and optionally working with browser storage.

Deliverable: A working product listing and cart application with no backend; code in your repo; demo to mentor.

Phase 3: TypeScript Fundamentals and Practice

Timeline: Days 9–10

Objective: Develop a strong foundation in TypeScript by learning core language concepts and applying them through structured problem-solving exercises.

Learning Platform: TypeHero — complete exercises from **Beginner** and **Learners** sections.

Learning Scope

- Basic TypeScript syntax
- Primitive types (string, number, boolean)
- Type inference
- Union and intersection types
- Arrays and tuples
- Objects and type aliases
- Functions and return types
- Optional and default parameters
- Enums

- Interfaces
- Generics (introductory)
- Utility types (basic)

Focus on type safety, correct type definitions, and avoiding `any` unless absolutely required.

POC: Product Listing & Cart System — TypeScript Implementation

What you will build: Take the JavaScript Product Listing & Cart system you built in Phase 2 (POC 3) and rewrite the logic in TypeScript so every variable, function parameter, and return value has a proper type.

What the system should do:

- Define an `interface` or `type` for `Product` (name, price, category, etc.)
- Define an `interface` or `type` for `CartItem` (product reference, quantity)
- Define an `interface` or `type` for `Cart` (list of cart items, total)
- Rewrite all functions (add to cart, update quantity, remove, calculate total) using these types
- Compile the TypeScript code to JavaScript using `tsc`
- There should be **zero** usage of `any` in your code

Deliverable: Type-safe cart logic that compiles cleanly; code in your repo with a `tsconfig.json`; README; Git history showing the refactoring.

Phase 4: React.js — Frontend Framework Training

Timeline: Days 11–17

Objective: Learn to build scalable, component-based front-end applications using React.js by completing certifications and implementing real-world projects.

Certification 1 — Beginner React

- **Course:** Free React JS Course
- **Provider:** Scaler Topics
- **Link:** https://www.scaler.com/topics/course/free-react-js-course/
- **Outcome:** Certificate of Completion
- **Topics:** Introduction to React and JSX, development environment setup, component architecture, props and state, forms and events, mini projects, basic hooks (`useState`, `useEffect`)

Certification 2 — Advanced React

- **Course:** Modern React with Redux, Router, Hooks and Auth
- **Provider:** Udemy
- **Link:** https://www.udemy.com/course/modern-react/
- **Outcome:** Certificate of Completion
- **Topics:** Core React, Redux, React Router, authentication and protected routes, hooks, end-to-end project implementation

POC 1: Product Listing & Cart System (React)

What you will build: Convert your JavaScript/TypeScript Product Listing & Cart application into a proper React application with reusable components and React state management.

What the system should do:

- Break the UI into React components: `ProductList`, `ProductCard`, `Cart`, `CartItem`
- Display a list of products — each product card shows name, price, description, and an "Add to Cart" button
- When "Add to Cart" is clicked, the product appears in the Cart section
- If the same product is added again, increase its quantity instead of creating a duplicate
- Inside the cart: user can increase/decrease quantity, remove items, and see the updated subtotal and total
- Show an "Empty cart" message when no items are present
- Use `useState` to manage product and cart state; use `useEffect` where needed
- **Optional:** Persist the cart to `localStorage` so it survives page refresh
- **Optional:** Add loading and error states to practice data-fetching patterns

Deliverable: A fully functional React Product & Cart application with clean component structure; code in your Git repository.

POC 2: Enhanced Todo & Productivity Tracker

What you will build: A feature-rich Todo application where users can add tasks with details, filter and sort them, and see a summary dashboard — all persisted in the browser.

What the system should do:

- Each task has: Title (required), Description, Priority (Low / Medium / High), Category (Work / Personal / Other), Due Date, and a Completed checkbox
- Users can: Add a new task, Edit an existing task, Delete a task, Search tasks by title
- Filter tasks by: status (completed/pending), priority, or category
- Sort tasks by: due date or priority
- Show a **summary panel** with: total tasks, completed count, pending count, and overdue count
- All tasks are saved in `localStorage` so they persist across page refreshes
- Validate inputs — for example, Title cannot be empty

Deliverable: A functional React Todo application; clean project structure; component-based architecture; code in your Git repository.

POC 3: Typing Speed Tester

What you will build: An interactive typing test where users type a displayed paragraph as fast and accurately as possible, and the app measures their speed and accuracy in real time.

What the system should do:

- Display a randomly selected or preset paragraph on the screen
- As the user types, highlight each character:

- **Green** for correct characters
- **Red** for incorrect characters
- **Underline or cursor highlight** for the current character
- Start a countdown timer when the user types the first character
- When time runs out, stop accepting input
- Show results: Words Per Minute (WPM), Characters Per Minute (CPM), and total mistakes
- Provide a "Restart" button to try again with the same or a new paragraph

Deliverable: An interactive typing speed test application built in React; timer and result display working correctly; polished UI; code in your Git repository.

Phase 5: Database Learning — SQL & PostgreSQL

Timeline: Days 18–19

Objective: Develop foundational database skills using SQL and PostgreSQL: write queries, design schemas, manipulate data, and understand transactions, triggers, functions, and stored procedures.

Learning Resources

- [SQL & Databases Full Course — freeCodeCamp]
- [PostgreSQL Course for Beginners — freeCodeCamp]

Core Topics

- **SQL Fundamentals:** `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`, aggregate functions (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`), `JOIN`s (INNER, LEFT, RIGHT, FULL), subqueries
- **Data Manipulation:** `INSERT`, `UPDATE`, `DELETE`
- **Schema Design:** Creating databases and tables, data types, constraints (`PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE`, `CHECK`), indexing
- **Transactions:** `BEGIN`, `COMMIT`, `ROLLBACK`; understanding atomicity; when to batch multiple operations into a single transaction
- **Triggers:** `BEFORE` and `AFTER` triggers on `INSERT`, `UPDATE`, `DELETE`; using triggers for audit logs and derived updates
- **Functions:** Creating custom SQL functions using `CREATE FUNCTION`; input parameters and return types; using functions in queries
- **Stored Procedures:** Creating procedures using `CREATE PROCEDURE`; difference between functions and procedures; calling procedures with `CALL`
- **Views:** Creating views using `CREATE VIEW`; using views to simplify complex queries
- **PostgreSQL Tools:** psql CLI, pgAdmin, or any preferred GUI tool

Practical Tasks — LeetCode SQL Problems

Complete the following 15 problems from LeetCode's Database section. These are chosen to align with the topics above.

#	LeetCode Problem	Link
1	Recyclable and Low Fat Products (#1757)	leetcode.com/problems/recyclable-and-low-fat-products
2	Big Countries (#595)	leetcode.com/problems/big-countries
3	Find Customer Referee (#584)	leetcode.com/problems/find-customer-referee
4	Combine Two Tables (#175)	leetcode.com/problems/combine-two-tables
5	Customers Who Never Order (#183)	leetcode.com/problems/customers-who-never-order
6	Duplicate Emails (#182)	leetcode.com/problems/duplicate-emails
7	Delete Duplicate Emails (#196)	leetcode.com/problems/delete-duplicate-emails
8	Employees Earning More Than Their Managers (#181)	leetcode.com/problems/employees-earning-more-than-their-managers
9	Rising Temperature (#197)	leetcode.com/problems/rising-temperature
10	Game Play Analysis I (#511)	leetcode.com/problems/game-play-analysis-i
11	Not Boring Movies (#620)	leetcode.com/problems/not-boring-movies
12	Second Highest Salary (#176)	leetcode.com/problems/second-highest-salary
13	Nth Highest Salary (#177)	leetcode.com/problems/nth-highest-salary
14	Rank Scores (#178)	leetcode.com/problems/rank-scores
15	Consecutive Numbers (#180)	leetcode.com/problems/consecutive-numbers

Hands-On Practice

- Create a sample schema with at least 3 related tables; insert 50+ records
- Write queries covering: filtering, aggregation, joins, subqueries, update, and delete with conditions
- Write one exercise demonstrating a **transaction** (e.g. transfer money between two accounts — both updates succeed or both fail)
- Write one exercise demonstrating a **trigger** (e.g. auto-update an `updated_at` timestamp whenever a row changes)
- Write one exercise creating a **function** (e.g. calculate order total given an order ID)
- Submit: SQL script files, learning log, and screenshots of completed LeetCode problems

---

Phase 6: Node.js Backend Development

Timeline: Days 20–29

Objective: Learn Node.js and Express.js fundamentals, then build a complete backend API as a single progressive project that covers user management, authentication, and product management.

Learning Resource (Mandatory)

- Node.js Course — Scaler Topics

Core Topics

- Node.js fundamentals: event loop, modules, file system, environment variables, error handling, async patterns
- Express.js: server setup, routing, middleware, request/response, error-handling middleware, REST best practices
- Database integration: SQL or NoSQL, CRUD operations, connection handling, validation

POC 1: Complete Backend API (Single Progressive Project)

What you will build: One backend application that you develop step-by-step. Each step builds on top of the previous one in the same codebase. By the end, you will have a fully functional API with user management, secure authentication, and product/cart features.

Step 1 — User Management CRUD

Build the foundation of your backend by creating a REST API to manage users.

- Set up a new Node.js + Express.js project with a clean folder structure: `routes/`, `controllers/`, `services/`, `models/`
- Create these API endpoints:
- `POST /api/users` — Create a new user (name, email, password)
- `GET /api/users` — Get all users
- `GET /api/users/:id` — Get a single user by ID
- `PUT /api/users/:id` — Update a user
- `DELETE /api/users/:id` — Delete a user
- Validate all input (e.g. email must be valid format, name cannot be empty)
- Return proper HTTP status codes (201 for created, 404 for not found, 400 for bad input, etc.)
- Connect to MongoDB or PostgreSQL
- Use environment variables for database connection string and port

Step 2 — Authentication & Authorization

Add secure login and role-based access to your existing user API.

- Add a `POST /api/auth/register` endpoint — hash the password (using bcrypt) before saving
- Add a `POST /api/auth/login` endpoint — verify credentials and return a JWT token
- Create an auth middleware that checks for a valid JWT on protected routes
- Add roles to users (e.g. `admin` and `user`)
- Protect certain routes so only logged-in users can access them
- Protect admin-only routes (e.g. delete user) so only admins can call them
- Store secrets (JWT secret, etc.) in environment variables — never hardcode them

Step 3 — Product & Cart CRUD

Add product management and cart functionality that aligns with your React Product Listing UI from Phase 4.

- Create Product API endpoints:

- `POST /api/products` — Create a new product (admin only)
- `GET /api/products` — Get all products (public)
- `GET /api/products/:id` — Get a single product
- `PUT /api/products/:id` — Update a product (admin only)
- `DELETE /api/products/:id` — Delete a product (admin only)
- Create Cart API endpoints:
- `POST /api/cart` — Add a product to the logged-in user's cart
- `GET /api/cart` — Get the current user's cart with product details
- `PUT /api/cart/:itemId` — Update quantity of a cart item
- `DELETE /api/cart/:itemId` — Remove an item from the cart
- `DELETE /api/cart` — Clear the entire cart
- Validate all input; handle errors with clear JSON responses
- Use async/await throughout

Deliverable: One Git repository containing the complete backend; README with API description, setup instructions, and endpoint list; Postman collection or screenshots; database schema documentation.

Phase 7: NestJS Backend Development

Timeline: Days 30–39

Objective: Learn NestJS framework and build backend APIs using its modular architecture, controllers, services, DTOs, validation pipes, and database integration.

Reference: [NestJS Full Course (YouTube)]

POC 1: User Authentication & Role-Based Access API

What you will build: A standalone authentication system using NestJS where users can register, log in, and access protected resources based on their role.

What the system should do:

- `POST /auth/register` — Register a new user with name, email, and password. Hash the password before storing.
- `POST /auth/login` — Verify email and password, return a JWT token on success.
- Create a JWT auth guard that protects routes — any request without a valid token gets rejected.
- Add roles (`admin`, `user`) and create a roles guard — certain endpoints should only be accessible to admins.
- Structure code using NestJS modules: `AuthModule`, `UsersModule`.
- Use DTOs with class-validator decorators (`@IsEmail`, `@IsString`, `@MinLength`) to validate all input.

POC 2: Product & Cart Management Backend

What you will build: A product catalog and shopping cart API using NestJS modules, following the same product model as your React and Node.js phases.

What the system should do:

- **Product endpoints:** Create, update, get all, get by ID, and delete products.
- **Cart endpoints:** Add product to cart, update quantity, remove item, get full cart with product details, and clear cart.
- Use NestJS modules: `ProductModule`, `CartModule` — each with its own controller and service.
- Define DTOs for every request body and validate using class-validator (`@IsString`, `@IsNumber`, `@IsNotEmpty`).
- Handle errors using NestJS exception filters — return proper HTTP status codes and structured JSON error responses.
- Connect to MongoDB (using Mongoose) or SQL (using TypeORM or Prisma).

POC 3: Order & Payment Backend System

What you will build: An order management system where users can place orders from their cart, track order status, and simulate a payment process.

What the system should do:

- `POST /orders` — Create an order from the current user's cart items. The order should include: list of products, quantities, total amount, and status set to "Pending".
- `GET /orders` — Get order history for the logged-in user.
- `GET /orders/:id` — Get details of a specific order.
- `PATCH /orders/:id/status` — Update order status (e.g. Pending → Paid → Shipped → Delivered). Admin only.
- `POST /payments/simulate` — Accept an order ID, simulate a payment (always succeeds or randomly fails), and update the order status to "Paid" on success.
- Use transactions where applicable (e.g. creating an order should clear the cart atomically).

POC 4: Notification & Email Service Backend

What you will build: A notification system that sends messages through different channels (in-app notifications and email) triggered by application events.

What the system should do:

- `POST /notifications` — Create a new in-app notification for a specific user (title, message, type).
- `GET /notifications` — Get all notifications for the logged-in user, with read/unread status.
- `PATCH /notifications/:id/read` — Mark a notification as read.
- Use NestJS `EventEmitter` to trigger notifications automatically when events happen (e.g. when an order is placed, send a notification to the user).
- **Optional:** Integrate a third-party email service (e.g. Nodemailer with a test SMTP) to send email notifications.
- Store notification history in the database with timestamps.

POC 5: Admin Dashboard & Reporting Backend

What you will build: A set of admin-only API endpoints that provide summary reports and data management capabilities.

What the system should do:

- `GET /admin/users` — List all users with pagination and optional search by name or email. Admin only.
- `GET /admin/orders/summary` — Return total orders, revenue, and order count grouped by status. Support date range filtering.
- `GET /admin/products/summary` — Return total products, most ordered products, and out-of-stock count.
- All admin endpoints must be protected by both JWT authentication and admin role authorization.
- Support query parameters for pagination (`page`, `limit`) and sorting (`sortBy`, `order`).
- Return well-structured JSON responses with pagination metadata.

Final POC — Full-Stack Integration

Timeline: Days 40–41

Objective: Connect the React frontend (built in Phase 4) with the Node.js backend (built in Phase 6) to create a working full-stack application.

What you will build: Take your Phase 4 React Product Listing & Cart application and connect it to the Phase 6 Node.js backend API so that products come from the real database and cart operations are handled by the server.

What the system should do:

- Configure the React app to call the backend API using an environment variable for the base URL (e.g. `REACT_APP_API_URL`)
- Implement user login in the React app — call the `/api/auth/login` endpoint, store the JWT token, and send it with every subsequent API request
- Replace the static product array with a real API call to `GET /api/products`
- Wire the "Add to Cart" button to call `POST /api/cart` on the backend
- Wire cart quantity updates and item removal to the corresponding backend endpoints
- Show loading spinners while API calls are in progress
- Show user-friendly error messages when API calls fail
- Ensure the full flow works end-to-end: login → browse products → add to cart → update cart → view cart total

Deliverable: A working full-stack application with React frontend calling real Node.js backend APIs; README with setup and run instructions for both frontend and backend; demo to mentor showing the complete user flow.

Summary

Phase	Timeline	Focus
1	Days 1–2	Git, GitHub & Object-Oriented Programming

Phase	Timeline	Focus
2	Days 3–8	JavaScript Fundamentals & POCs
3	Days 9–10	TypeScript Fundamentals and Practice
4	Days 11–17	React.js — Frontend Framework Training
5	Days 18–19	Database Learning — SQL & PostgreSQL
6	Days 20–29	Node.js Backend Development
7	Days 30–39	NestJS Backend Development
—	Days 40–41	Final POC — Full-Stack Integration

This document is the final internship curriculum for the 3-month Full Stack Engineer Internship Program. Companion document: `Intern_Progress_Tracking_Document_Full_Stack_Internship`.